

C++

C++ Essentials Part 1

CPE-20-01 · Nivel Básico



Sintaxis



Control



Funciones



Arrays



Struct



Strings

```
01 #include <iostream>
02 #include <string>
03
04 using namespace std;
05
06 struct Estudiante {
07     string nombre;
08     int notas[3];
09 };
10
11 int main() {
12     Estudiante e{"Ana", {18, 16, 20}};
13     cout << "Nombre: " << e.nombre << endl;
14     cout << "Notas: ";
15     for (int i = 0; i < 3; ++i) cout << e.notas[i] << " ";
16     return 0;
17 }
```



**Si tengo una duda y no pregunto por no pasar
vergüenza 5 minutos...
es muy probable que esa duda me haga pasar
vergüenza muchas veces en mi vida**

**POR FAVOR, preguntad todo lo que se os ocurra...
AQUÍ NOS HEMOS REUNIDO PARA APRENDER!!!**



Contexto

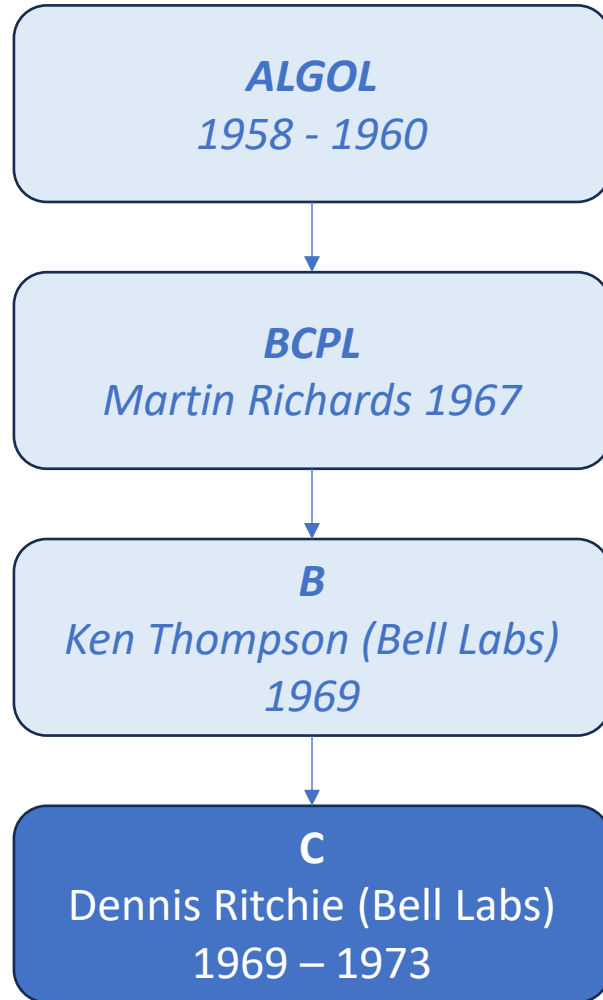


Un poco de historia

1954-1957		FORTRAN	Orientado al cálculo científico
1958		LISP	Introdujo la programación funcional
1958-1960		ALGOL	Popularizó la programación estructurada
1959: Grace Hopper		COBOL	Diseñado para aplicaciones empresariales. Gran interacción con DB
Década del 60		Simula	Introdujo conceptos como clases, objetos y herencia
1972: Dennis Ritchie		C	Nacieron para resolver problemas de programación de sistemas: SO, compiladores, controladores y aplicaciones donde importan mucho el rendimiento y el acceso directo al hardware.
1986: Bjarne Stroustrup		C++	



Antecedentes de C



C surgió durante el desarrollo de **Unix**.

Ken Thompson y Dennis Ritchie necesitaban un lenguaje que combinara:

- La eficiencia y el control del ensamblador.
- Estructuras de programación de alto nivel.
- La posibilidad de trasladar programas entre máquinas diferentes.

En 1973, gran parte del núcleo de Unix fue reescrito en C. Esto fue revolucionario: hasta entonces, los sistemas operativos solían escribirse casi completamente en ensamblador y quedaban muy ligados a un hardware concreto.



C

- Se suele describir como un lenguaje de nivel medio.
- Compilación a código nativo muy eficiente.
- Tipado estático.
- Programación procedural o procedimental.
- Uso explícito de punteros.
- Gestión manual de memoria.
- Una biblioteca estándar pequeña.
- Gran portabilidad entre plataformas.

C se utilizó para desarrollar Unix y posteriormente Linux, compiladores, bases de datos, sistemas embebidos y numerosas bibliotecas. También influyó en la sintaxis de lenguajes como C++, Java, C#, JavaScript, PHP, Go y Rust.



C++ suele elegirse cuando, además del rendimiento, se necesitan abstracciones para construir sistemas complejos

Comenzó incorporando clases, constructores, destructores, herencia y polimorfismo.

Con el tiempo añadió muchas otras formas de programación:

- Programación procedural heredada de C.
- Programación orientada a objetos.
- Programación genérica mediante plantillas.
- Programación funcional mediante funciones lambda.
- Programación de bajo nivel y gestión directa de recursos.
- Abstracciones de alto nivel sin renunciar al rendimiento.



C / C++

- C++ nació a partir de C y conserva gran parte de su sintaxis
- NO es simplemente “C con objetos”
- Actualmente son dos lenguajes diferentes, con estándares y filosofías propias.
- Una interfaz mínima con el hardware.
- Máxima portabilidad entre plataformas pequeñas.
- Compatibilidad binaria sencilla.
- Desarrollo de sistemas operativos y microcontroladores.
- Motores de videojuegos.
- Navegadores.
- Aplicaciones de escritorio.
- Sistemas financieros.
- Robótica.
- Simulación y cálculo científico.
- Software en tiempo real.



Ambos lenguajes ofrecen un alto grado de control,
pero ese control implica responsabilidad.

El programador debe comprender aspectos como
la memoria, la duración de los objetos, los tipos y
la gestión de recursos.

Aprender C y C++ proporciona una visión
especialmente profunda de cómo funciona un
programa y cómo se relaciona con el hardware.



Mi primer programa

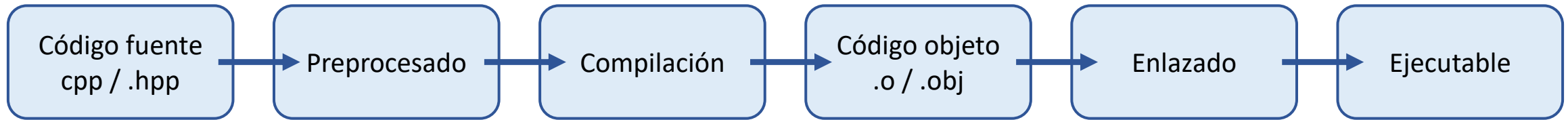


Algunas definiciones

Elemento	Función
Editor	Escribir y navegar por el código.
Compilador	Detectar errores y traducir cada unidad de traducción a código objeto.
Enlazador	Combinar objetos y bibliotecas, resolviendo símbolos.
Depurador	Detener la ejecución e inspeccionar variables y pila de llamadas.
Sistema de construcción	Automatizar opciones, archivos y dependencias.
IDE (Integrated Development Environment)	Integra todas las herramientas.



Creación del ejecutable



- El preprocesador expande directivas como `#include` y macros
- El compilador analiza una unidad de traducción y genera código objeto.
- El enlazador resuelve las referencias entre objetos y bibliotecas.



El primer programa

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Bienvenidos al curso C++" << std::endl;
5      return 0;
6  }
```

```
PS C:\cursos> g++ .\hola-mundo.cpp -o hola-mundo
PS C:\cursos> .\hola-mundo.exe
Bienvenidos al curso C++
PS C:\cursos>
```

```
C:\cursos>g++ hola-mundo.cpp -o hola-mundo

C:\cursos>hola-mundo
Bienvenidos al curso C++

C:\cursos>|
```

```
ubuntu@THINKPAD-PABLO:~/cpp/prueba$ g++ hola-mundo.cpp -o hola-mundo
ubuntu@THINKPAD-PABLO:~/cpp/prueba$ ./hola-mundo
Bienvenidos al curso C++ en Linux
ubuntu@THINKPAD-PABLO:~/cpp/prueba$ █
```



Mi primer programa

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Bienvenidos al curso C++" << std::endl;
5     return 0;
6 }
```

indica que es una
directiva del preprocesador

Nombre de la función

Tipo de retorno de la función main

Retorna 0 al S.O. (sin error)

Utilizar el espacio de nombres std
Evitamos usar: std::cout y str::endl

Las dos formas de comenzar un
bloque son correctas

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     cout << "Bienvenidos al curso C++" << endl;
8     return 0;
9 }
```



El programa línea a línea

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Bienvenidos al curso C++" << std::endl;
5      return 0;
6  }
```

- **#include <iostream>** hace visibles las declaraciones necesarias para los streams estándar
- **main** es una función especial. **int** indica que la función devuelve un entero
- **{}** las llaves delimitan el bloque y el punto y coma **;** finaliza la sentencia simple
- **std::cout** es el stream de salida estándar; **<<** inserta datos en ese stream
- **std::endl** obliga a vaciar el buffer agregando un salto de línea
- **return 0;** finaliza la función devolviendo un 0 (Fin del programa sin error)



Sintaxis

EXPRESIONES:

Conjunto de elementos o tokens que son evaluados para devolver un resultado o realizar una tarea.

TOKENS:

- Identificadores: nombre de variables, estructuras, clases, funciones, constantes, ...
- Palabras claves: Identificadores reservados del lenguaje.
- Literales: valores que pueden tomar las variables, son valores constantes.
- Operadores: Indican la evaluación que se debe aplicar a un objeto o dato.
- Separadores: Indican al compilador donde se ubican los diferentes elementos del código:
`{},:,;`

Los comentarios, saltos de línea, espacios vacíos y de tabulación, no se consideran tokens, ya que el compilador los elimina.



Sintaxis

EXPRESIONES:

Conjunto de elementos o tokens que son evaluados para devolver un resultado o realizar una tarea.

TOKENS:

- Identificadores: nombre de variable, ...
- Palabras claves: Identificadores reservados, ...
- Literales: valores que pueden tomar
- Operadores: Indican la evaluación
- Separadores: Indican al compilador la estructura del código:
{}, ::, ;

```
//...  
while (i == 0) {  
    resu = a + b;  
    i = 1;  
    //bucle que no sirve para nada  
}  
//...
```

Los comentarios, saltos de línea, espacios vacíos y de tabulación, no se consideran tokens, ya que el compilador los elimina.



Sintaxis

- En C++ se hace distinción entre mayúsculas y minúsculas (case-sensitive)
- Las instrucciones simples terminan en punto y coma
- Los bloques de código se delimitan mediante llaves {}



Identificadores

Por ej. nombres de variables

- Secuencia de letras, dígitos y el carácter guion bajo _
- Debe comenzar con una letra o _ (evitando iniciar con doble _)
- Letra: cualquier carácter Unicode que describa una letra: ñ, á, Π, ä, etc.
- No se puede utilizar palabras reservadas.

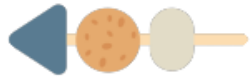
Ej.:

nombre, saldoActual, PORCENTAJE_IVA, año,
paísDeNacimiento, valor01



Convenciones de nombres

Precio sin impuestos



kebab-case (skewer-case)

precio-sin-impuestos



camelCase

precioSinImpuestos



PascalCase o UpperPascalCase

PrecioSinImpuestos



snake_case

precio_sin_impuestos



SCREAMING_SNAKE_CASE

PRECIO_SIN_IMPUESTOS



Convenciones y recomendaciones

C++ no impone una convención concreta para nombrar variables, funciones o tipos. Sin embargo, utilizar una convención coherente facilita la lectura y el mantenimiento del código. Debe darse prioridad a las normas establecidas por el proyecto o el equipo.

- Variables y funciones snake_case o camelCase
- Tipos (clases, estructuras, enumeraciones y alias) PascalCase
- Constantes SCREAMING_SNAKE_CASE (depende mucho de la política del proyecto)



Resumen de convenciones

Elemento	Convención	Ejemplo
Programas ejecutables	kebab-case	inventory-server
Archivos y directorios	snake_case	user_repository.cpp
Espacios de nombres	snake_case	payment_processing
Clases y estructuras	PascalCase	PaymentService
Enumeraciones	PascalCase	ConnectionState
Valores de enumeración	PascalCase	ConnectionState::Connected
Alias de tipos	PascalCase	UserId
Funciones y métodos	snake_case	calculate_total()
Variables y parámetros	snake_case	item_count
Objetos	snake_case	payment_service
Atributos privados	snake_case_	connection_count_
Constantes conceptuales	kPascalCase	kMaximumRetries
Booleanos	prefijo semántico	is_ready
Parámetros de plantilla	PascalCase	ValueType
Macros	UPPER_SNAKE_CASE	PROJECT_VERSION



Comentarios

Junto al código fuente de un programa, se pueden insertar comentarios

- De una línea (//...) se ignora todo hasta encontrar un salto de línea
- De bloque (/*...*/) se ignora todo lo que está entre /* y */

```
1  #include <iostream>
2
3  //Este es nuestro primer programa
4  int main() {
5      /*
6          Muestra un mensaje
7          de bienvenida
8          por la consola
9      */
10     std::cout << "Bienvenidos al curso C++" << std::endl;
11     return 0;
12 }
```



Variables locales de tipo fundamental

Una variable local, al definirla, crea la entidad. Si se utiliza, tendrá un valor indeterminado.

<code>int edad;</code>	Definida, pero sin valor inicial determinado. No está inicializada.
<code>int alumnos = 20;</code>	Inicialización por copia. Operador =
<code>int grupos(3);</code>	Inicialización directa. Paréntesis
<code>int aulas{4}</code>	Inicialización uniforme. Llaves
<code>int vacio{};</code>	Inicialización a cero. Llaves sin valor



ASCII y codificaciones Unicode

Codificación	Unidad de código	Tamaño por carácter	
ASCII	7 bits (normalmente 1 byte)	1 byte	Codifica 128 símbolos: letras inglesas, números, signos y controles.
UTF-8 *	8 bits = 1 byte	1–4 bytes	Unicode de longitud variable. ASCII mantiene la misma representación.
UTF-16 *	16 bits = 2 bytes	2 o 4 bytes	Usa una unidad para la mayoría de caracteres comunes y dos para otros.
UTF-32	32 bits = 4 bytes	4 bytes	Cada valor Unicode ocupa una unidad fija; es sencillo, pero consume más memoria.

* La unidad de código tiene tamaño fijo; un carácter puede necesitar una o varias unidades.



Tipos de datos fundamentales

- Tipos de datos
 - int
 - float
 - double
 - char
 - bool
- Modificadores
 - short (puede modificar a int)
 - long (puede modificar a int y double)
 - unsigned(puede modificar a los enteros y char)



Tipos de datos fundamentales

Familia	Tipos principales	Observaciones
Booleano	bool	Valores false y true. Acepta conversiones contextuales desde números y punteros. Cero/no cero.
Caracteres	char, signed char, unsigned char, char8_t, char16_t, char32_t	char ocupa un byte de C++; no equivale necesariamente a un carácter Unicode completo.
Enteros con signo	short, int, long, long long	Cada tipo puede escribirse con signed explícito. El desbordamiento con signo es comportamiento indefinido.
Enteros sin signo	unsigned short, unsigned int, unsigned long, unsigned long long	Aritmética módulo 2^N ; útil en bits, pero peligrosa al mezclar con enteros con signo.
Reales	float, double, long double	Representación aproximada; no comparar resultados calculados con igualdad exacta salvo casos controlados.
Sin valor	void	Se usa para funciones que no devuelven valor y en algunos contextos de punteros.



Cadenas

- Las cadenas (string) no son tipos fundamentales, son objetos
- Las trataremos de forma similar a los tipos fundamentales
- Debemos utilizar la directiva **#include <string>**
- “string “ no es una palabra reservada
- Inicialización
 - `string nombre = “Carmen”;`
 - `string nombre(“Carmen”);` //Estilo orientado a objetos
 - Cualquiera de las dos formas es equivalente y pueden ser utilizadas indistintamente
 - `string cliente = nombre;`
 - `string cliente(nombre);`



Operadores

Aritméticos	+ - * / %
	++ --
Relacionales	> >= < <= == !=
Lógicos	&& ! &
Desplazamiento y lógicos de bits	<< >>
	& ^ ~
Asignación	= += -= *= /= %=
	&= = ^= <<= >>=
Condicional	?:



Precedencia operadores

! ~ (type) ++ -- + - operadores unarios
* / %
+ - operadores binarios
<< >>
< <= > >=
== !=
&
|
&&
||
= += -= *= /= %= &= ^= |= >>= <<=



Stream de entrada salida por defecto

- `std::cout` salida estándar
- `std::cin` entrada estándar
- `<<` operador de inserción de flujo
- `>>` operador de extracción de flujo (están sobrecargados)

```
5      int cant_alumnos{};  
6  
7      std::cout << "Presentes: ";  
8      std::cin >> cant_alumnos;  
9  
10     std::cout << "Han asistido " << cant_alumnos << " alumnos." << std::endl;
```



Entrada por consola

- El operador >> extrae datos formateados y deja en el stream (buffer de teclado) el primer carácter que no pertenece al dato. Por ej. el salto de línea.
- Muy útil para leer tipos fundamentales, pero por ejemplo para leer cadenas que contengan espacios no funciona, ya que lee solo la primera palabra, el resto de la cadena queda en el buffer de teclado.
- Además, ignora siempre los primeros espacios, tabuladores y saltos de línea, útil si se leer varios tipos fundamentales.
- Para leer cadenas (frases) debemos utilizar

`std::getline(std::cin, variable_str)`

`std::getline(std::cin >> std::ws, variable_str)`

`std::ws` descarta todos los caracteres de separación iniciales (espacios, ‘\n’, etc.)



Biblioteca cmath - #include <cmath>

Función	Operación	Ejemplo	Resultado ej.
<code>std::sqrt(x)</code>	Raíz cuadrada	<code>std::sqrt(16.0)</code>	4
<code>std::pow(x, y)</code>	Potencia x^y	<code>std::pow(2.0, 3.0)</code>	8
<code>std::abs(x)</code>	Valor absoluto	<code>std::abs(-5)</code>	5
<code>std::round(x)</code>	Redondeo al entero más cercano	<code>std::round(3.7) / round(-3, 7)</code>	4 / -4
<code>std::floor(x)</code>	Redondeo hacia abajo	<code>std::floor(3.9) / floor(-3, 9)</code>	3 / -4
<code>std::ceil(x)</code>	Redondeo hacia arriba	<code>std::ceil(3.1) / ceil(-3, 1)</code>	4 / -3
<code>std::sin(x)</code>	Seno (expresado en radianes)	<code>std::sin(numbers::pi / 2.0)</code>	1
<code>std::cos(x)</code>	Coseno (expresado en radianes)	<code>std::cos(numbers::pi)</code>	-1
<code>std::log(x)</code>	Logaritmo natural (base e)	<code>std::log(numbers::e)</code>	1
<code>std::log10(x)</code>	Logaritmo decimal (base 10)	<code>std::log10(1000)</code>	3



Inferencia de tipo (auto)

auto unidades = 12;

- auto pide al compilador que deduzca el tipo a partir del inicializador.
- El tipado sigue siendo estático, no dinámico

```
7      auto cantidad = 45; //int
8      auto precio = 9.95; // double
9      auto letra = 'A'; // char
```

```
16     std::vector<std::pair<std::string, int>> datos;
17     auto elemento = datos.begin();
```

```
20     std::vector<std::pair<std::string, int>>::iterator elemento = datos.begin();
```



Constantes (const)

- Const impide modificar el objeto a través de ese nombre después de inicializarlo.

`const <tipo_dato> <identificador> = <valor>;`

```
const double IVA = 0.21;
```

```
const int LADOS_TRIANGULO = 3;
```



Literales y tipos

- Se puede decir que en C++ $5 \neq 5.0$, aunque $5 == 5.0$ de verdadero
- Los literales, además de su valor, tienen TIPO
 - 5 es int
 - 5.0 es double

```
10      auto num1 = 10;      // int
11      auto num2 = 10U;     // unsigned int
12      auto num3 = 10L;     // long
13      auto num4 = 10UL;    // unsigned long
14      auto num5 = 10LL;    // long long
15      auto num6 = 10ULL;   // unsigned long long
16
17      auto num7 = 3.14;     // double
18      auto num8 = 3.14F;    // float
19      auto num9 = 3.14L;    // long double
```



Literales

• 42	literal entero	tipo int
• 3.14	literal de punto flotante	tipo double
• 'a'	literal de carácter	tipo char
• "hola"	literal de cadena	tipo const char[5]
• true	literal booleano	tipo bool
• nullptr	literal de puntero nulo	tipo nullptr_t



Literales numéricos

ENTEROS

LITERAL	BASE	VALOR (decimal)
101	Decimal (10)	101
0b101	Binario (2)	5
0101	Octal (8)	65
0x101	Hexadecimal (16)	257

```
9      int num1 = 5;  
10     int num2 = -578;  
11     int num3 = +5;  
12     int num4 = 12'000'000;  
13     int num5 = 0b0000'0100;
```

REALES

LITERAL	NOTACIÓN	VALOR
15300.0	Normal	15.300,0
1.53e+4	Científica	15.300,0

```
24     double d1 = 0.5;  
25     double d2 = .5;  
26     double d3 = -.5;  
27     double d4 = +.5;  
28     double d5 = .7E5;  
29     double d6 = .3e-5;
```



Secuencias de escape

\"	"
\'	'
\b	Carácter backspace (BS)
\t	Tabulador (TAB)
\v	Tabulador vertical
\n	Avance de Línea (LF)
\r	Retorno de Carro (CR)
\f	Avance de Página (FF)

\uXXXX	Carácter UNICODE, XXXX hexadecimal
\0 - \377	Secuencia octal – Caracteres ASCII
\xhh..	Secuencia hexadecimal
\0	Carácter nulo
\\	\
\a	Alerta sonora (depende de la consola)

R"(...)" raw string (cadena sin procesar). Ignora las secuencias de escape

R"(c:\documentos\trabajo\tmp)"



bool y condiciones

- C++ permite usar valores convertibles a bool en condiciones. Cero y punteros nulos se interpretan como false; los demás valores numéricos, como true.

```
int cantidad = ...;
//...
if (cantidad != 0) {
    std::cout << "Hay elementos\n"
}
```

```
int cantidad = ...;
//...
if (cantidad) {
    std::cout << "Hay elementos\n"
}
```



Estructuras de control



Sentencias

► Simples

Toda sentencia simple finaliza con ;

► Compuestas o bloque

Conjunto de sentencias delimitadas por {}

```
8      double a = 1, b = -1, c = -2;
9      double x1, x2, discr;
10
11     //Calculo raices
12     {
13         discr = sqrt(b * b - 4 * a * c);
14         if(discr >= 0) {
15             x1 = (- b + discr) / (2*a);
16             x2 = (- b - discr) / (2*a);
17         }
18     }
19
20     cout << "x1 = " << x1 << '\n';
21     cout << "x2 = " << x2 << '\n';
```



Estructuras de control – if / if else

Condicional Simple

```
if (expresión_lógica)  
    sentencia;
```

Condicional Doble

```
if (expresión_lógica)  
    sentencia_1;  
else  
    sentencia_2;
```



Cortocircuito y condiciones seguras

a = 0;

//...

si (a != 0 AND <condición desconocida>)

Se ejecuta A

else

Se ejecuta B

```
if (divisor != 0 && dividendo/divisor > 10) {  
    // La división sólo se evalúa cuando divisor != 0  
}
```



Estructuras de control - switch

Condicional Múltiple

```
switch (pivote) {  
    case valor_1:  
        sentencia_1;  
    case valor_2:  
        sentencia_2;  
    ...  
    case valor_n:  
        sentencia_n;  
    default:  
        sentencia;  
}
```

El pivote puede ser:

- int
- char
- short
- long
- enum

- Los valores de cada case deben ser constantes en tiempo de compilación.
- El default es opcional



Estructuras de control - switch

```
8      string nombre_dia;
9      int dia;
10     // ...
11     switch (dia) {
12     |     case 1: nombre_dia = "lunes";
13     |         break;
14     |     case 2: nombre_dia = "martes";
15     |         break;
16     |     case 3: nombre_dia = "miercoles";
17     |         break;
18     |     // ...
19     |     case 7: nombre_dia = "domingo";
20     |         break;
21     |     default: nombre_dia = "dia incorrecto";
22     |
23     }
24     cout << dia << ": " << nombre_dia << '\n';
```

e deben ser
e compilación.



Estructuras de control – while / do-while

Estructuras iterativas

```
while (expresión_lógica)  
    sentencia;
```

```
bool mas_datos = true;  
while (mas_datos) {  
    // ...  
}
```

```
do  
    sentencia;  
while (expresión_lógica);
```

```
bool mas_datos = true;  
do {  
    // ...  
} while (mas_datos);
```



Estructuras de control - for

Estructuras iterativas

```
for (sentencia_inicializ; expresión_lógica; sentencia_actualiz)  
    sentencia;
```

```
for (int i = 1; i <= 8; i++) {  
    // se repetirá 8 veces  
    std::cout << i << '\n';  
}
```

```
int i;  
for (i = 1; i <= 8; i++) {  
    // se repetirá 8 veces  
    std::cout << i << '\n';  
}
```



Estructuras de control - ruptura

Sentencias de ruptura

- **break**
- **continue**
- **return**

Rompe la ejecución de la estructura de control.
La ejecución continúa en la siguiente instrucción de la estructura.

Rompe la ejecución de la iteración actual de la estructura de control.
La ejecución continúa en la siguiente iteración.

Sale de la función actual y devuelve el control al llamador.
Devuelve el valor necesario en funciones.



Funciones



Funciones

- Las funciones en C++ son **subalgoritmos**
- Un **subalgoritmo** es una parte independiente de un algoritmo que realiza una tarea específica y puede reutilizarse
 - **Funciones:** ejecutan acciones y devuelven un resultado
 - **Procedimientos:** ejecutan acciones y no devuelven un resultado
- En C++ existen funciones que retornan un valor y otras que no retornan nada



Declaración de una función – Prototipo de la función

- Una función antes de ser invocada debe estar **declarada**.

```
tipoDevuelto nombre_funcion([lista parámetros]);
```

- El compilador debe conocer la declaración (**prototipo**) antes de poder interpretar cualquier invocación a la función.
- La declaración de una función informa al compilador de su **nombre**, **el tipo del valor devuelto** y **los tipos de los parámetros** (si existen).



Definición de una función

- Si la declaración además contiene el cuerpo, significa que la función está definida.

```
tipoDevuelto nombre_funcion([lista parámetros]) {  
    //instrucciones;  
    [return valor;]  
}
```

tipoDevuelto

- Tipos fundamentales
- Objetos
- void
- Si se omite, asume int

Lista parámetros

- Tipo nombre
- Separados por ,
- Si no recibe, puede
puede declarar **void**

return valor;

- Obligatorio si tipoDevuelto no es void
- El tipo de valor debe ser compatible con tipoDevuelto



Ejemplo función

```
1  # include <iostream>
2
3  using namespace std;
4
5  // Declaración
6  // Prototipo de la función
7  double cuadrado(double);
8
9  int main(void) {
10     double cuad = cuadrado(7.);
11     cout << cuad << endl;
12
13     double valor = 9;
14     cout << cuadrado(valor) << endl;
15
16     return 0;
17 }
18
19 // Definición
20 double cuadrado(double num) {
21     return num * num;
22 }
```

```
1  # include <iostream>
2
3  using namespace std;
4
5  // deficiión
6  double cuadrado(double num) {
7     return num * num;
8 }
9
10 int main(void) {
11     double cuad = cuadrado(7.);
12     cout << cuad << endl;
13
14     double valor = 9;
15     cout << cuadrado(valor) << endl;
16
17     return 0;
18 }
```



Arrays



Arrays nativos

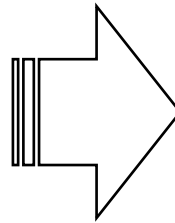
- ¿Por qué una estructura de datos?
- Son estructuras de datos estáticas (se debe conocer la cantidad de elementos en tiempo de compilación)
- Todos los elementos que se almacenan deben ser del mismo tipo
- No se puede crear un array nativo en una función para retornarlo!!
- `int nums[5]`
 - `nums` es una variable que permite almacenar 5 valores de tipo `int`, o dicho formalmente, `nums` es un array de `int` de tamaño 5
 - Los elementos se acceden a través de un índice, desde 0 a 4
 - `nums[0] = 17;`
 - `std::cout << "Primer elemento: " << nums[0];`



Arrays

- `int nums[5]`
 - Igual que con las variables escalares, a cada elemento del array se le deberá asignar un valor (inicializar) para poder consultarlo.
- Ejemplo (uso incorrecto)

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6
7      int nums[5];
8      for (int i = 0; i < 5; i++)
9          cout << nums[i] << endl;
10     cout << "-----" << endl;
11
12
13     return 0;
14 }
15
```



```
-2116379104
32762
-926943264
31
-926943248
-----
```

Arrays – inicialización y recorrido

`int nums[5] = {1, 2, 4, 8, 16}; //inicializador`

`int nums[5] = {}, //todos los elementos se inicializan en 0`

`int nums[5] = {3, 6}; //nums[0] vale 3, nums[1] vale 6 y el resto 0`

`int nums[] = {1, 2, 3}; //equivale a int nums[3] = {1, 2, 3};`

Recorrer el array

```
for (int i = 0; i < 5; i++) {  
    std::cout << nums[i] << '\n';  
}
```



for basado en rango – (for-each)

```
for (tipo elemento : contenedor) {  
    //usar elemento  
}
```

Permite recorrer cualquier objeto que represente un rango, por ejemplo, los arrays.

```
8      string dias_laborables[] = {"lunes", "martes", "miercoles", "jueves", "viernes"};
```

```
10     for (int i = 0; i < 5; i++)  
11         cout << dias_laborables[i] << endl;
```

```
14     for (string dia : dias_laborables)  
15         cout << dia << '\n';
```



Array fijo <array> (C++ moderno)

- Un array fijo, igual que los nativos, es una secuencia cuyo número de elementos se debe conocer en tiempo de compilación y no cambia durante la ejecución.
- Su almacenamiento también es contiguo.
- Se comporta como un objeto, puede copiarse, conocer el tamaño `size()`, acceder a sus elementos con `at()` y mas funcionalidad.

```
#include <array>

6 //creación e inicialización
7 std::array<int, 5> decenas{10, 20, 30, 40, 50};
8
9 //inicialización a 0
10 std::array<double, 3> coordenadas{}; // {0., 0., 0.}
11
12 //rellenarlo todo con un valor concreto
13 std::array<double, 5> valores;
14 valores.fill(-1); // {-1.0, -1.0, -1.0, -1.0, -1.0}
15
16 //cuidado!
17 std::array<double, 5> otro{-1}; // {-1.0, 0.0, 0.0, 0.0, 0.0}
18
19 //inferencia de tipo y tamaño
20 std::array datos{15, 30, 45}; // std::array<int, 3>
```

```
22 //se puede copiar
23 auto copia = decenas;
24
25 //conocemos el tamaño
26 std::cout << decenas.size() << '\n'; //5
27 std::cout << copia.size() << '\n'; //5
28 std::cout << datos.size() << '\n'; //3
29
30 //acceso a los elementos
31 double valor = valores.at(2); //comprueba los límites
32 double otro_valor = valores[2]; //no controla los límites
33
34 std::cout << "Primer elemento: " << decenas.front() << '\n';
35 std::cout << "Ultimo elemento: " << decenas.back() << '\n';
36
37 //recorrido
38 for (double v : valores)
39     std::cout << v << '\n';
40
41 for (int i = 0; i < datos.size(); i++)
42     std::cout << datos.at(i) << '\n';
```



Paso de parámetros y retorno de arrays

- Se puede crear un <array> en una función y retornarlo para que lo utilice el llamador.
- Para declarar un <array> como parámetro hay que establecer el tamaño.
- A partir de C++ 20 existe span, que permite declarar un parámetro sin indicar el tamaño del array

```
1  #include <iostream>
2  #include <array>
3
4  using namespace std;
5
6  // retornar un nuevo array
7  array<int, 5> crea_array_5() {
8      return {17, -158, 895, 0, 12};
9  }
10
11 int main() {
12     auto datos = crea_array_5();
13     for (auto dato : datos){
14         cout << dato << endl;
15     }
16
17     return 0;
18 }
```

```
1  #include <iostream>
2  #include <array>
3
4  using namespace std;
5
6  void recibe(array<int, 5> v) {
7      for (auto dato : v) {
8          cout << dato << endl;
9      }
10 }
11
12 int main() {
13     array<int, 5> arr = {1,2,3,4,5};
14     recibe(arr);
15
16     return 0;
17 }
```

```
1  #include <iostream>
2  #include <array>
3  #include <span>
4
5  using namespace std;
6
7  // A partir de C++ 20 existe span
8  void muestra(span<const int> v) {
9      cout << "[";
10     for (size_t i = 0; i < v.size() - 1; i++) {
11         cout << v[i] << ", ";
12     }
13     cout << v[v.size() - 1] << "]" << endl;
14 }
15
16 int main() {
17     array<int, 5> arr = {1,2,3,4,5};
18     muestra(arr);
19
20     int arr2[]{12, 14, 16};
21     muestra(arr2);
22     return 0;
23 }
```



Matrices

Crear una matriz de 7 filas y 3 columnas

$$M = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

```
8      double m[7][3] = {};  
9  
10     for (int i = 0; i < 7; i++) {  
11         for (int j = 0; j < 3; j++)  
12             cout << m[i][j] << "\t";  
13         cout << endl;  
14     }
```

```
17     const int filas = 7;  
18     const int cols = 3;  
19     double m[filas][cols] = {};  
20  
21     for (int i = 0; i < filas; i++) {  
22         for (int j = 0; j < cols; j++)  
23             cout << m[i][j] << "\t";  
24         cout << endl;  
25     }
```

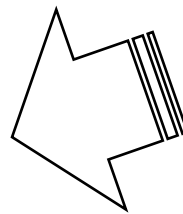


Matrices inicializadas

Crear una matriz de 5 filas y 4 columnas con valores

$$M = \begin{pmatrix} 0 & 1 & 7 & 9 \\ 1 & 3 & 6 & 3,5 \\ 0,75 & 0,66 & 9 & 0 \\ 3 & 2 & 7,5 & 6 \\ 4 & 5,5 & 3 & 0 \end{pmatrix}$$

0	1	7	9
1	3	6	3.5
0.75	0.66	9	0
3	2	7.5	6
4	5.5	3	0



```
28
29
30 const int filas = 5;
31 const int cols = 4;
32 double m[filas][cols] = {
33     {0., 1., 7., 9.},
34     {1., 3., 6., 3.5},
35     {0.75, .66, 9., 0.},
36     {3., 2., 7.5, 6.},
37     {4., 5.5, 3., 0.},
38 };
39
40 for (int i = 0; i < filas; i++) {
41     for (int j = 0; j < cols; j++)
42         cout << m[i][j] << "\t";
43     cout << endl;
```



Arrays multidimensionales (arrays de arrays de arrays)

- Necesitamos controlar la cantidad de huéspedes alojados en un hotel con las siguientes características:
 - Tres edificios: A, B y C
 - Cada edificio tiene 15 plantas.
 - Cada planta tiene 20 habitaciones.

```
int huespedes[3][15][20] = {};
```

- C++ no limita el número de dimensiones de un array



Funciones – parte 2



Funciones: ámbito de variables y efectos secundarios

```
1  #include <iostream>
2
3  using namespace std;
4
5  int global = 1;
6
7  void una_funcion();
```

```
9  int main(void) {
10     int local_main = 2;
11
12     if (local_main > 0) {
13         int local_if = 3;
14         cout << local_if << endl;
15         cout << local_main << endl;
16         cout << global << endl;
17     }
18
19     // cout << local_if << endl;
20     cout << local_main << endl;
21     cout << global << endl;
22
23     for (int i = 0; i < 5; i++)
24         cout << i << endl;
25
26     // cout << i << endl;
27
28     {
29         int local_bloque = 5;
30     }
31     // cout << local_bloque << endl;
32
33     una_funcion();
34
35     return 0;
36 }
```

```
38 void una_funcion() {
39     int local_funcion = 5;
40     cout << local_funcion << endl;
41     cout << global << endl;
42     // cout << local_main << endl;
43 }
```



Funciones: Paso de parámetros

Paso de
parámetros

Por valor

Por referencia

Por valor

Si una función declara un parámetro de tipo T, el parámetro es un objeto local de la función que se inicializa a partir del valor del argumento utilizado en la llamada

El valor del argumento se copia en T

Por referencia

Si una función declara un parámetro de tipo referencia T&, el parámetro se enlaza con el argumento correspondiente y las operaciones efectuadas mediante él se aplican al objeto referenciado

T es un alias del argumento



Paso de Parámetros

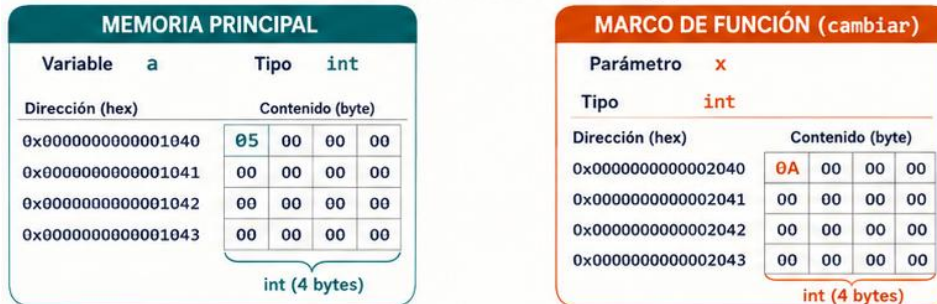
PASO POR VALOR

```
void cambiar(int x) { x = 10; }  
int a = 5;  
cambiar(a);
```

ANTES DE LLAMAR A cambiar(a)



DESPUÉS DE EJECUTAR cambiar(a)

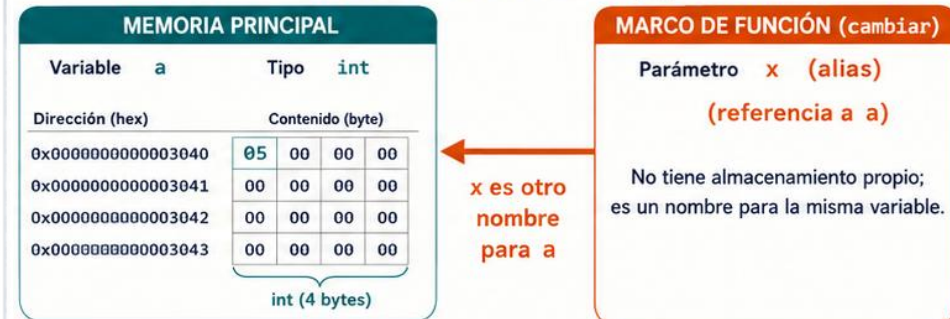


✓ Dos variables independientes · Modificar **x** no cambia **a**

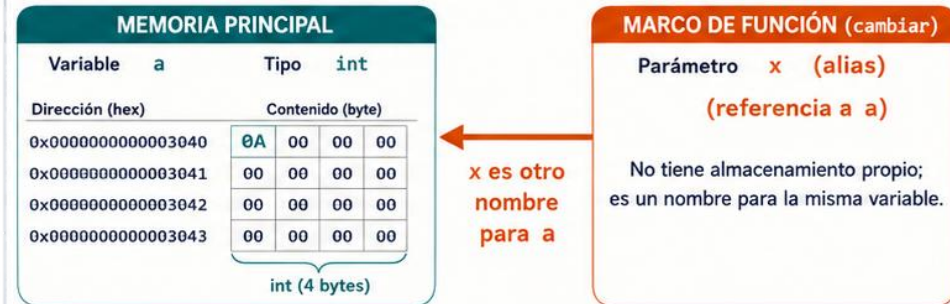
PASO POR REFERENCIA

```
void cambiar(int& x) { x = 10; }  
int a = 5;  
cambiar(a);
```

ANTES DE LLAMAR A cambiar(a)



DESPUÉS DE EJECUTAR cambiar(a)



✓ Misma variable · Modificar **x** cambia **a**



Parámetros predeterminados

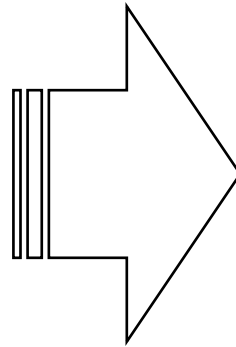
- En la declaración de un parámetro se le puede asignar un valor por defecto
- Si no se pasa un argumento para ese parámetro, utiliza el valor por defecto
- Pueden existir varios parámetros predeterminados, pero deben ser los últimos en la lista

```
1  #include <iostream>
2
3  using namespace std;
4
5  double suma_iva(double importe, double iva = 0.21) {
6      return importe * (1 + iva);
7  }
8
9  int main(void) {
10     double total_factura = 1000.0;
11     cout << total_factura << ": " << suma_iva(total_factura) << endl;
12     cout << total_factura << ": " << suma_iva(total_factura, 0.1) << endl;
13     return 0;
14 }
```



Funciones inline

```
5  inline int duplica(int valor) {  
6      return valor * 2;  
7  }  
8  
9  int main(void) {  
10     int var = 10;  
11     var = duplica(var);  
12     var = duplica(var);  
13     var = duplica(var);  
14     var = duplica(var);  
15     cout << var << endl;  
16     return 0;  
17 }
```



```
9  int main(void) {  
10     int var = 10;  
11     var = var * 2;  
12     var = var * 2;  
13     var = var * 2;  
14     var = var * 2;  
15     cout << var << endl;  
16     return 0;  
17 }
```



Function signature

```
double cuadrado(double num) {  
    return num * num;  
}
```

Function signature: **cuadrado(double)**

El compilador lo utiliza para identificar qué función se está invocando

nombre + tipos de parámetros + espacio de nombres al que pertenece



Sobrecarga de funciones

- Mecanismo que permite disponer de varias funciones con un mismo nombre.
- Las funciones sobrecargadas deben tener diferente “signature”
- Las funciones sobrecargadas se distinguen:
 - Por la cantidad de parámetros
 - Por los tipos de cada parámetro

```
5  int mayor(int, int);  
6  int mayor(int, int, int);  
7  double mayor(double, double);  
8  double mayor(double, double, double);
```



Struct (estructuras)



Estructuras

- Es una estructura de datos que nos permite agrupar bajo un nombre, datos de distintos tipos.
- Necesitamos registrar los datos de alumnos que realizan un curso on-line, su nombre, tiempo conectado y ultimo capitulo realizado.

```
6 struct Alumno {  
7     string nombre;  
8     double tiempo;  
9     int ultimo_capitulo;  
10 };
```

Declaración. Crea un tipo de datos, no crea ninguna variable.

```
14 Alumno alu1;  
15  
16 struct Alumno alu2;
```

Creación de variables

```
19 alu1.nombre = "Ana Maria";  
20 alu1.tiempo = 3.5;  
21 alu1.ultimo_capitulo = 2;
```

Acceso a los campos. Operador de acceso .



Estructuras

Declaración

```
6 struct Fecha {  
7     int dia, mes, anyo;  
8 };  
9  
10 struct Alumno {  
11     string nombre;  
12     double tiempo;  
13     int ultimo_capitulo;  
14     Fecha fecha_nacimiento;  
15 };
```

Inicialización

```
28 //Con inicializador  
29 Alumno alu3 = {"Jorge", 1.5, 1, {15, 6, 2001}};
```

```
31 //Con inicializador - si están todos los datos se pueden omitir las llaves de la fecha  
32 Alumno alu4 = {"Marta", 2., 2, 21, 2, 1992};
```

```
34 //Con inicializador sin asignación  
35 Alumno alu5 {"Fernando", 4.25, 5, 10, 2, 1998};
```

```
37 //Con inicializador vacío  
38 Alumno alu6 {}; // o ={};
```

```
40 //Array de estructura  
41 Alumno alumnos[100];  
42 alumnos[0] = {"Isabel", 5.25, 7, 1, 6, 2001};
```



Estructuras y arrays como parámetros

```
void func1(Alumno alu);
```

alu es pasado por valor: **“se copia todo el contenido del original a alu”**

Si no necesitamos modificar el contenido, lo pasamos por **referencia constante**

```
void func1(const Alumno& alu);
```

```
void func2(int[] v, int cant);
```

Los arrays tradicionales son pasados por referencia (veremos punteros). Si se modifica el valor de v, modifica el original. Para evitar la modificación del original lo pasamos **constante**

```
void func2(const int[] v, int cant);
```

```
void func3(std::array<int, 100> v);
```

Con std::array **SI se produce la copia completa**. Igual que con struct, **referencia constante**

```
void func3(const std::array<int, 100>& v);
```

```
void func4(const std::span<int>& v);
```

Con std::span **NO se produce la copia completa**, span es una vista de los datos.

```
void func4(const std::span<const int>& v);
```



Punteros



Punteros

- Un puntero es un tipo de dato.
- En una variable de tipo puntero, es capaz de almacenar una dirección de memoria.
- Guardan información sobre la ubicación de otros datos.

```
int a;
```

Crea una variable de tipo int, asocia una dirección de memoria con el identificador **a** y puede almacenar un entero en “4 bytes”

```
int *ptr;
```

Crea una variable de tipo int* (puntero a int), asocia una dirección con **ptr** y puede almacenar una dirección de memoria donde se aloje un int

```
a = 5;
```

Almacena 5 en la dirección de memoria asociada con **a**

```
ptr = &a;
```

Almacena la dirección de memoria donde se aloja **a** en la variable **ptr**, utiliza el operador de referencia **&**



Punteros

C++

```
int a = 5;  
int* ptr = &a;
```

a

Variable a

valor: 5

ptr

Puntero ptr

valor: dirección de a

Dirección (cada byte)

Contenido (1 byte)

0x00007FFC12341040	05	a (int, 4 bytes)
0x00007FFC12341041	00	
0x00007FFC12341042	00	
0x00007FFC12341043	00	
...	...	
0x00007FFC12341050	0x00007FFC12341040	ptr (int*, 8 bytes)
0x00007FFC12341051	00	
0x00007FFC12341052	00	
0x00007FFC12341053	00	
0x00007FFC12341054	00	
0x00007FFC12341055	00	
0x00007FFC12341056	00	
0x00007FFC12341057	00	



Punteros

```
ptr = NULL;
```

Puntero nulo, no apunta a ninguna dirección. Podría utilizarse `ptr = 0`, pero puede crear confusiones. `NULL` es una macro. Tenemos que incluir `<iostream>` o `<cstring>`

```
cout<<*ptr;
```

Con ***ptr** accedemos al valor de la variable donde apunta `ptr`. Desreferenciamos `ptr`

```
*ptr = 17;
```

Modificamos el valor de la variable donde apunta **ptr**

Desreferenciar un puntero no debería usarse nunca.



sizeof

- Es un operador unario prefijo
- Indica cuantos bytes de memoria ocupa su argumento
- Ej: sizeof(char)
- Su argumento puede ser un tipo, una variable o un literal
- Sólo si el argumento es un tipo los paréntesis son obligatorios

```
9      int num = 77;
10     char letras[121];
11
12     cout << "sizeof num: " << sizeof num << endl;
13     cout << "sizeof 1589: " << sizeof 1589 << endl;
14     cout << "sizeof (1589): " << sizeof(1589) << endl;
15     cout << "sizeof(double): " << sizeof(double) << endl;
16     cout << "sizeof(long long): " << sizeof(long long) << endl;
17     cout << "sizeof(long double): " << sizeof(long double) << endl;
18     cout << "sizeof 'A': " << sizeof 'A' << endl;
19     cout << "sizeof letras: " << sizeof letras << endl;
```

```
4      sizeof num: 4
4      sizeof 1589: 4
4      sizeof (1589): 4
8      sizeof(double): 8
8      sizeof(long long): 8
16     sizeof(long double): 16
1      sizeof 'A': 1
121    sizeof letras: 121
```



Punteros y arrays

El identificador de un array, en realidad es un puntero al primer elemento del array

```
int *ptr, arr[5];
```

Declaración de un puntero a int **ptr** y un array de int de 5 elementos **arr[0]**

```
ptr = &arr[0];
```

Guardamos la dirección del primer elemento del array **&arr[0]** en **ptr**

```
ptr == arr
```

Verdadero

```
*ptr == arr[0]
```

Verdadero



Aritmética de punteros

```
int i, *p1, *p3, arr[5] = {10, 20, 30};  
p1 = arr;
```

- Sumar un entero a un puntero y obtener otro puntero `p3 = p1 + 2;`
p3 apuntará a **arr[2]** es decir la dirección que está 8 bytes posterior a p1
la aritmética tiene en cuenta el tipo de dato
- Restar un entero a un puntero y obtener otro puntero `p3 = p3 - 1;`
p3 apuntará a **arr[1]** es decir la dirección que está 4 bytes anterior a p3
- Restar un puntero de otro y obtener un entero `i = p3 - p1;`
i valdrá 1
- Comparar dos punteros (== o !=)



Ordenación



Métodos de ordenación

- Hay muchas formas (algoritmos) de ordenar un array
- Los algoritmos se diferencian por:
 - Complejidad
 - Tiempo en función del tamaño
 - Consumo de memoria
- Para comparar la eficiencia de un algoritmo se utiliza
 - Complejidad computacional $O(n)$
 - Es la cantidad de instrucciones simples que ejecuta el algoritmo para realizar la tarea
 - Se mide en función de la cantidad de elementos a procesar (n)
 - Se puede medir en distintas situaciones
 - Mejor caso
 - Promedio
 - Peor caso



Métodos de ordenación

- Inserción (Insertion sort)
 - Cada elemento se inserta en el subconjunto que ya está ordenado
 - $O(n^2)$
- Selección (Selection sort)
 - Se va seleccionando el mínimo elemento y se lo ubica en su lugar
 - $O(n^2)$
- Burbuja (Bubblesort)
 - $O(n^2)$
- Quick Sort (Quicksort)
 - $O(n \log n)$



Punteros – parte 2



Punteros amorfos

void *ptr;

- Un puntero amorfo puede apuntar a un valor de cualquier tipo.
- No puede desreferenciarse (*ptr). El compilador no sabría cómo interpretar el valor.
- Cuando se conoce el tipo, puede convertirse en otro puntero del tipo necesario.
- Se pueden usar para:
 - Interfaces genéricas escritas en C
 - Funciones que trabajan con datos de distintos tipos
 - Bibliotecas antiguas y operaciones de bajo nivel
 - Gestión manual de memoria
- En C++ hay otros mecanismos mas seguros



Memoria bajo demanda

- C++ permite que el programador gestione la memoria
- Es útil cuando no se conoce el tamaño de los datos a procesar
- **new** permite reservar memoria y construir el objeto
- **delete** destruye el objeto y liberar esa memoria

```
8      int cant;  
9      cout << "Cantidad de valores: ";  
10     cin >> cant;  
11  
12     int* numeros = new int[cant];  
13  
14     // Uso del array...  
15  
16     delete[] numeros;  
17     numeros = nullptr;
```



Arrays de punteros

- Si necesitamos crear una matriz (array en dos dimensiones) con tamaños a definir en tiempo de ejecución, no podemos utilizar:

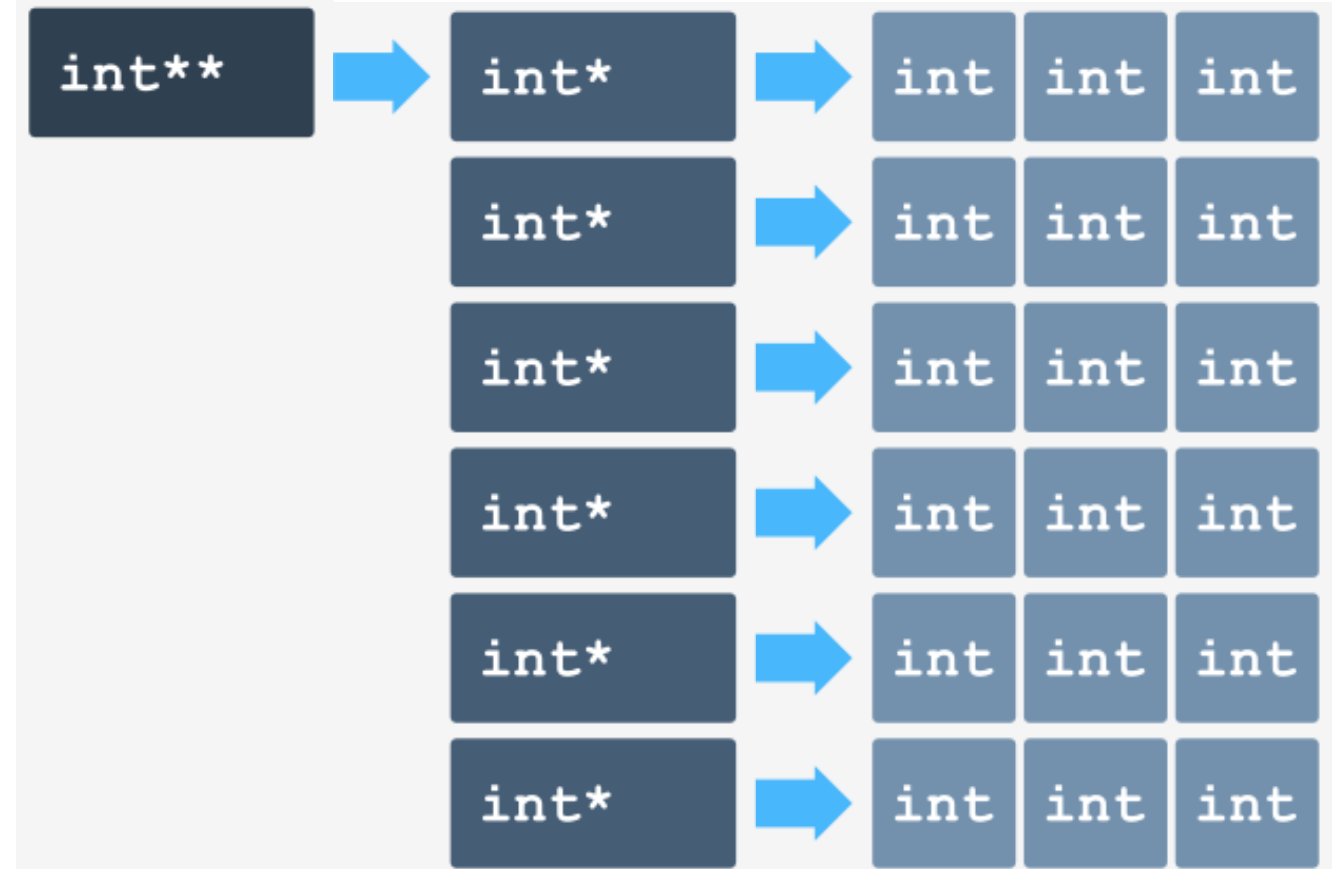
~~`new int[filas][cols]`~~

- Debemos crear varios punteros, es decir, cada puntero apunta al primer elemento de cada fila
- Esos punteros los almacenamos en un array de punteros

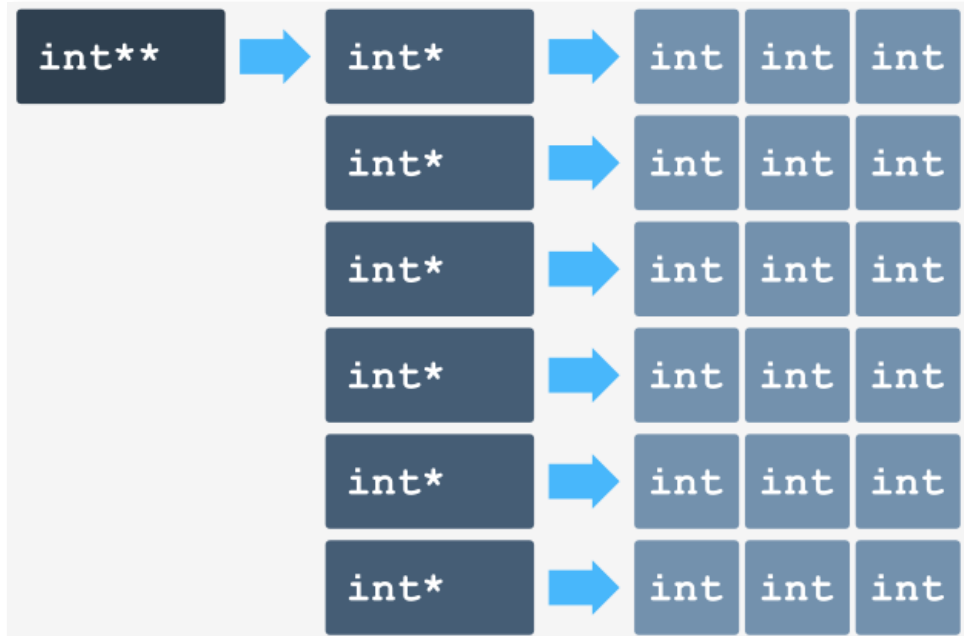


Arrays de punteros

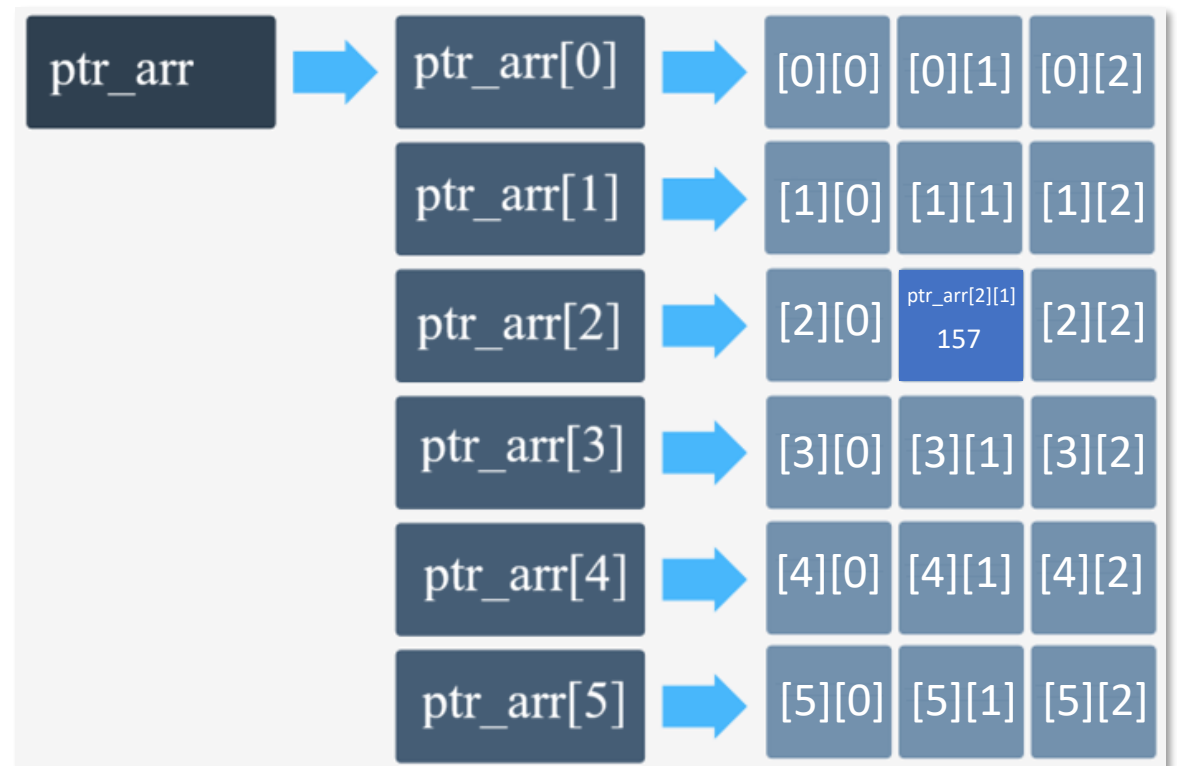
```
int **ptr_arr;  
ptr_arr = new *int[filas]; //filas = 6  
for(int f=0; f<filas; f++)  
    ptr_arr[f] = new int[cols]; //cols = 3
```



Arrays de punteros

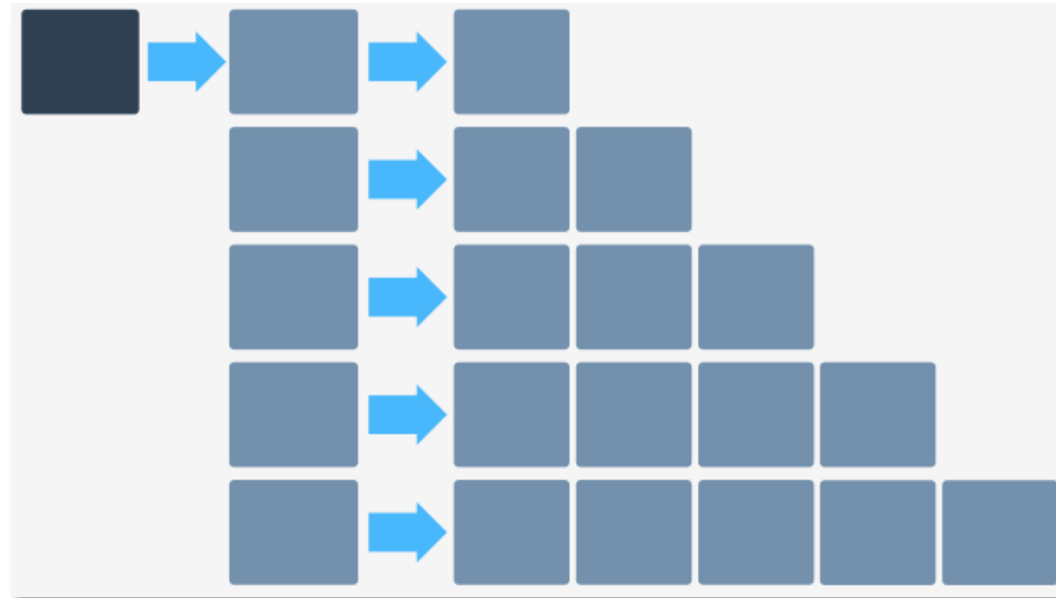


`ptr_arr[2][1] = 157;`



Arrays de punteros

- La ventaja de estos arrays es que, a diferencia de los arrays ordinarios, cada fila puede tener una longitud diferente
- Resulta útil en algoritmos que no necesitan utilizar el array completo, sino solo una parte de él
- Por ejemplo, para representar matrices triangulares



Conversiones de tipos type casting



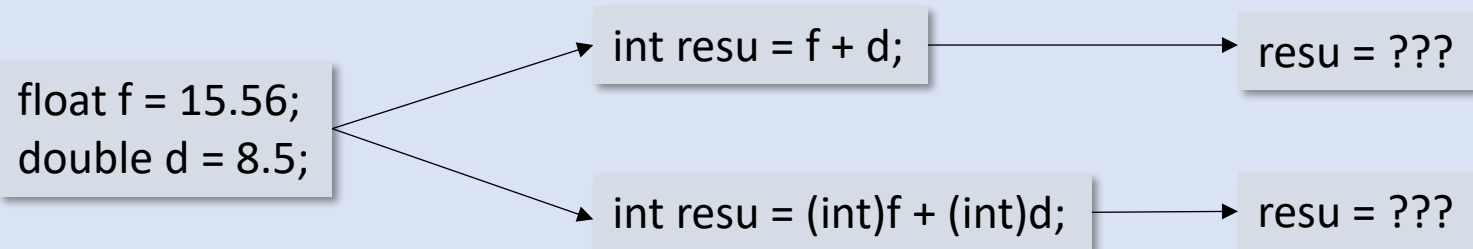
Conversiones

Conversiones implícitas

- Con ganancia
`short s = 14568;`
`int i = s;`
`s == i => verdadero`
- Con pérdida
`i = 55000;`
`s = i;`
`s == i => falso`

Conversiones explícitas

- Estilo C
 - `(nuevo_tipo) expresión;`
`short s = 14568;`
`int i = (int) s;`
- Notación funcional (C++)
 - `nuevo_tipo(expresión);`
`i = int(s);`



Conversiones

Conversiones implícitas

- Con ganancia
`short s = 14568;`
`int i = s;`
`s == i => verdadero`
- Con pérdida
`i = 55000;`
`s = i;`
`s == i => falso`

Conversiones explícitas

- Estilo C
 - `(nuevo_tipo) expresión;`
`short s = 14568;`
`int i = (int) s;`
- Notación funcional (C++)
 - `nuevo_tipo(expresión);`
`i = int(s);`

`float f = 15.56;`
`double d = 8.5;`

`int resu = f + d;`

resu = 24

`15,56 + 8.5 => 24.06 => 24`

`int resu = (int)f + (int)d;`

resu = 23

`15,56 + 8.5 => 15 + 8 => 23`



Promociones

```
short s = 27;  
int i = 55;  
auto resu = s + i;
```

¿De qué tipo es resu?

- Dentro de una expresión, los valores y variables se promocionan al tipo de mayor intervalo.
- Todas las promociones se realizan de acuerdo con estas reglas:
 - Los datos de tipo **char** o **short** se convierten a **int siempre** (promoción entera)
 - Si en una expresión entera aparece un **long long**, todos se promocionan a **long long**
 - Si aparece un **float** y no hay ningún double, todos se promocionan a **float**
 - Si aparece un **double** y no hay ningún long double, todos se promocionan a **double**
 - Finalmente, si aparece un **long double**, todos se promocionan a **long double**

int >> long long >> float >> double >> long double



string (cadenas)



Cadenas

- Las cadenas (string) no son tipos fundamentales, son objetos
- Las trataremos de forma similar a los tipos fundamentales
- Se pueden representar como un array de char (C). En C++ utilizamos **string**
- Debemos utilizar la directiva **#include <string>**
- “string” no es una palabra reservada
- Inicialización
 - `string nombre = “Carmen”;`
 - `string nombre(“Carmen”);` //Estilo orientado a objetos
 - Cualquiera de las dos formas es equivalente y pueden ser utilizadas indistintamente
 - `string cliente = nombre;`
 - `string cliente(nombre);`



string

- Concatenación (+) y (+=)
- Entrada por consola
 - **cin** trata al espacio y algunos otros caracteres como delimitadores
 - Para ingresar una frase
`getline(cin, nombre_variable_string);`
- Asignación
`string nuevo = cadena;`
 - Hace una copia (clon) del objeto. Existen dos objetos con el mismo contenido
- Comparación (comparan el contenido del objeto)
 - `==` `!=` `<` `<=` `>` `>=`

`nuevo == cadena`

Verdadero tienen el mismo contenido

`&nuevo == &cadena`

Falso son dos objetos diferentes (alojados en distintos lugares)



string – usando objetos (funciones miembros)

Los objetos, además de tener un contenido tienen funcionalidad integrada que interactúa con su contenido

- Para acceder a su funcionalidad utilizamos el operador de acceso: punto (.)

**size()
length()**

Devuelven la cantidad de caracteres.
Ambas hacen exactamente lo mismo

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main(void) {
7
8      string s1 = "Hola que tal";
9      string s2 = "Adios";
10
11     cout << s1.size() << endl;
12     cout << s2.size() << endl;
13
14     s2.append(" querido amigo!");
15     cout << s2 << endl;
16
17     return 0;
18 }
```

append(basic_string s)

Añade texto (s) al final del string

12

5

Adios querido amigo!



string – compare()

int compare(basic_string)

- basic_string: es el nombre genérico, string es un alias
- Compare retorna un entero al comparar el string con el que pasamos por parámetro

s1.compare(s2)

- 0: si s1 == s2
- < 0: si s1 < s2
- > 0: si s1 > s2

```
6 void muestra_compare(string s1, string s2) {  
7     string mensaje;  
8     int comp = s1.compare(s2);  
9     if (comp == 0)  
10         mensaje = " es igual a ";  
11     else if (comp < 0)  
12         mensaje = " es anterior a ";  
13     else  
14         mensaje = " es posterior a ";  
15     cout << s1 << mensaje << s2 << endl;  
16 }
```

```
18 int main(void) {  
19     string n1 = "Juan";  
20     string n2 = "Ana";  
21     string n3 = "Pedro";  
22     string n4 = "Ana";  
23     string n5 = "ana";  
24  
25     muestra_compare(n1, n2);  
26     muestra_compare(n1, n3);  
27     muestra_compare(n2, n4);  
28     muestra_compare(n2, n5);  
29     muestra_compare(n1, n5);  
30  
31     return 0;  
32 }
```

Juan es posterior a Ana
Juan es anterior a Pedro
Ana es igual a Ana
Ana es anterior a ana
Juan es anterior a ana



Sub cadenas (substring)

`basic_string substr(size_type pos = 0, size_type count = npos)`

- `size_type`: entero sin signo que representa posiciones o cantidades
- `npos`: constante que representa 'ninguna posición', 'no encontrado' o 'final de la cadena'

```
8  int main(void) {
9      string s1 = "Hoy puede ser un gran día, planteatelo así";
10     cout << "s1.size(): " << s1.size() << endl;
11
12     cout << s1.substr(0, 25) << endl;
13     cout << s1.substr(25, 2) << endl;
14     cout << s1.substr(27) << endl;
15     cout << s1.substr();
16
17     return 0;
18 }
```

s1.size(): 42

Hoy puede ser un gran día

‘_’
planteatelo así

Hoy puede ser un gran día, planteatelo así



string – mas funciones miembro

<code>empty()</code>	Indica si la cadena está vacía
<code>clear()</code>	Elimina todo el contenido
<code>at(pos)</code>	Devuelve el carácter de una posición (controla que exista pos) También se puede usar <code>[pos]</code> (no controla pos)
<code>front()</code> y <code>back()</code>	Retorna el primer / último carácter
<code>push_back(char)</code> y <code>pop_back()</code>	Añade / elimina un carácter al final
<code>insert(pos, str)</code>	Inserta un texto (str) en una posición (pos)
<code>erase(pos, cant)</code>	Elimina cant caracteres a partir de pos
<code>replace(pos, cant, str)</code>	Sustituye desde pos cant caracteres por str
<code>find(str)</code>	Busca str y devuelve su posición. Si no lo encuentra, npos
<code>assign(cant, char)</code>	Rellena la cadena con cant caracteres char. Reemplaza lo anterior



Intercambio de strings

- Para intercambiar el contenido de dos cadenas
 - Variable auxiliar
 - swap(otro_str)
 - Para cadenas largas, es mucho mas óptimo swap
 - También existe
 std::swap(var1, var2)
- Puede intercambiar cualquier tipo de dato

```
6  int main(void) {
7      string hola = "Hola, buenos días";
8      string adios = "Adios, hasta luego";
9
10     cout << "hola: " << hola << endl;
11     cout << "adios: " << adios << endl;
12
13     string aux = hola;
14     hola = adios;
15     adios = aux;
16     cout << "hola: " << hola << endl;
17     cout << "adios: " << adios << endl;
18
19     hola.swap(adios);
20     cout << "hola: " << hola << endl;
21     cout << "adios: " << adios << endl;
22
23     return 0;
24 }
```



Funciones útiles de conversión

Función	Conversión
<code>stoi(texto)</code>	<code>string</code> → <code>int</code>
<code>stol(texto)</code>	<code>string</code> → <code>long</code>
<code>stof(texto)</code>	<code>string</code> → <code>float</code>
<code>stod(texto)</code>	<code>string</code> → <code>double</code>
<code>to_string(valor)</code>	número → <code>string</code>

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main(void) {
7
8      string edad_str = "37";
9      string precio_str = "159.50";
10
11     int edad = stoi(edad_str);
12     double precio = stod(precio_str);
13
14     cout << edad << endl;
15     cout << precio << endl;
16
17     return 0;
18 }
```



Espacios de nombre



Espacio de nombres (namespace)

- Un espacio de nombres es un ámbito en el que un nombre determinado tiene un significado claro y no ambiguo

espacio_nombre::nombre_entidad

- Es un nombre cualificado

`std::cout std::string std::endl`

- `::` es el operador de resolución de ámbito
- Usar un espacio de nombres seleccionado:
`using namespace espacio_nombre;`
El alcance de `using` son las llaves donde ha sido declarado



namespace

- Crear un espacio de nombres

```
namespace curso {  
    //variables  
    //funciones  
    //etc  
}
```

- Los espacios de nombres se pueden ampliar en cualquier fuente

```
namespace std {  
    int algo = 456;  
}
```

- Crear un alias

```
namespace xx = std;
```

```
1  #include <iostream>  
2  #include <string>  
3  
4  namespace mio = std;  
5  
6  int main(void) {  
7      std::string tema = "namespace";  
8      mio::cout << tema << mio::endl;  
9      return 0;  
10 }
```

